Real-world (config + logging together) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: its own folder, `practice/config_logging_wiring/`, because it needs more than one file:

```
practice/config_logging_wiring/
├── exceptions.py        MissingConfigError
├── config.py            load_config() -> Config
├── logging_setup.py     get_logger(name)
├── main.py              wires the two together
└── .env                 you create this; never committed
```

Run it: `cd practice/config_logging_wiring && python main.py` — from inside the folder, so `from config import load_config` finds the file sitting next to it.

Builds on earlier exercises: `get_logger()` (from the logger exercise) and the custom-error pattern (re-written here as `MissingConfigError`). All four files are shown in full below, so this folder runs on its own.

Solution

Create these four files in `practice/config_logging_wiring/`, plus a `.env` next to them containing `OPENAI_API_KEY="sk-test123"` and `LOG_LEVEL=DEBUG`.

```
# practice/config_logging_wiring/exceptions.py
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# practice/config_logging_wiring/config.py
import os
from dataclasses import dataclass

from dotenv import load_dotenv

from exceptions import MissingConfigError
```

```
@dataclass
class Config:
    openai_api_key: str
    log_level: str

def load_config() -> Config:
    load_dotenv()
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise MissingConfigError("OPENAI_API_KEY is missing")
    return Config(
        openai_api_key=api_key,
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# practice/config_logging_wiring/logging_setup.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setFormatter(
            logging.Formatter(
                "%(asctime)s %(name)s %(levelname)s %(message)s"
            )
        )
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# practice/config_logging_wiring/main.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    # load config first — fail fast if a required value is missing
    config = load_config()
except MissingConfigError as e:
```

```
    print(f"Startup failed: {e}")
    sys.exit(1)

logger = get_logger(__name__)
logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output (terminal), with the `.env` above:

```
Loaded config with log level: DEBUG
```